

EXPERIMENT 11

MODES OF DES

AIM

To implement the different modes of operation of DES, namely ECB, CBC, CFB, OFB, and CTR, and observe their encryption and decryption processes.

MODES OF DES:

1. ECB – Electronic Codebook

- Each plaintext block is encrypted independently.
- The same plaintext block produces the same ciphertext block.
- It is simple but does not provide good security for repeated data.

$P1 \rightarrow \text{DES} \rightarrow C1$

$P2 \rightarrow \text{DES} \rightarrow C2$

$P3 \rightarrow \text{DES} \rightarrow C3$

2. CBC – Cipher Block Chaining

- Each plaintext block is XORed with the previous ciphertext block before encryption.
- An Initialization Vector (IV) is used for the first block.
- It provides better security than ECB.

$P1 \oplus IV \rightarrow \text{DES} \rightarrow C1$

$P2 \oplus C1 \rightarrow \text{DES} \rightarrow C2$

$P3 \oplus C2 \rightarrow \text{DES} \rightarrow C3$

3. CFB – Cipher Feedback

- The previous ciphertext is used as input to the DES encryption function.
- It can operate like a stream cipher.
- No separate padding is required for complete blocks.

$\text{DES}(IV) \oplus P1 \rightarrow C1$

$\text{DES}(C1) \oplus P2 \rightarrow C2$

4. OFB – Output Feedback

- The output of DES is repeatedly fed back as the input.
- It generates a keystream that is XORed with plaintext.
- Transmission errors do not propagate.

IV $\rightarrow$ DES $\rightarrow$ Output1
Output1 $\rightarrow$ DES $\rightarrow$ Output2
Output1 $\oplus$ P1 $\rightarrow$ C1
Output2 $\oplus$ P2 $\rightarrow$ C2

5. CTR – Counter Mode

- A counter value is encrypted using DES.
- The resulting keystream is XORed with the plaintext.
- The counter is incremented for every block.

Counter1 $\rightarrow$ DES $\rightarrow$ Keystream1
Counter2 $\rightarrow$ DES $\rightarrow$ Keystream2

P1 $\oplus$ Keystream1 $\rightarrow$ C1
P2 $\oplus$ Keystream2 $\rightarrow$ C2

PROGRAM:

```
#include <stdio.h>
```

```
#include <stdint.h>
```

```
/* ----- DES TABLES ----- */
```

```
int IP[64] = {  
    58,50,42,34,26,18,10,2,  
    60,52,44,36,28,20,12,4,  
    62,54,46,38,30,22,14,6,  
    64,56,48,40,32,24,16,8,  
    57,49,41,33,25,17,9,1,  
    59,51,43,35,27,19,11,3,  
    61,53,45,37,29,21,13,5,  
    63,55,47,39,31,23,15,7  
};
```

```
int FP[64] = {  
    40,8,48,16,56,24,64,32,  
    39,7,47,15,55,23,63,31,  
    38,6,46,14,54,22,62,30,  
    37,5,45,13,53,21,61,29,  
    36,4,44,12,52,20,60,28,  
    35,3,43,11,51,19,59,27,  
    34,2,42,10,50,18,58,26,  
    33,1,41,9,49,17,57,25  
};
```

```
int PC1[56] = {  
    57,49,41,33,25,17,9,  
    1,58,50,42,34,26,18,  
    10,2,59,51,43,35,27,  
    19,11,3,60,52,44,36,  
    63,55,47,39,31,23,15,  
    7,62,54,46,38,30,22,  
    14,6,61,53,45,37,29,  
    21,13,5,28,20,12,4  
};
```

```
int PC2[48] = {  
    14,17,11,24,1,5,  
    3,28,15,6,21,10,  
    23,19,12,4,26,8,  
    16,7,27,20,13,2,  
    41,52,31,37,47,55,
```

```
    30,40,51,45,33,48,  
    44,49,39,56,34,53,  
    46,42,50,36,29,32  
};
```

```
int E[48] = {  
    32,1,2,3,4,5,  
    4,5,6,7,8,9,  
    8,9,10,11,12,13,  
    12,13,14,15,16,17,  
    16,17,18,19,20,21,  
    20,21,22,23,24,25,  
    24,25,26,27,28,29,  
    28,29,30,31,32,1  
};
```

```
int P[32] = {  
    16,7,20,21,29,12,28,17,  
    1,15,23,26,5,18,31,10,  
    2,8,24,14,32,27,3,9,  
    19,13,30,6,22,11,4,25  
};
```

```
int SHIFTS[16] = {  
    1,1,2,2,2,2,2,2,  
    1,2,2,2,2,2,2,1  
};
```

```
/* S-Boxes */
```

int S[8][4][16] = {

{
{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
{0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
{4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
{15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}
},

{
{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
{3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
{0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
{13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}
},

{
{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
{13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
{13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
{1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}
},

{
{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
{13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
{10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
{3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}

},

{
{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
{14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
{4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
{11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}
},

{
{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
{10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
{9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
{4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}
},

{
{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
{13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
{1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
{6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}
},

{
{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
{1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
{7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
{2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}
}

```
};
```

```
/* ----- DES FUNCTIONS ----- */
```

```
uint64_t permute(uint64_t input, int table[],  
                 int n, int inputBits)
```

```
{
```

```
    uint64_t output = 0;
```

```
    int i;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        output <<= 1;
```

```
        output |= (input >>
```

```
                    (inputBits - table[i])) & 1ULL;
```

```
    }
```

```
    return output;
```

```
}
```

```
uint32_t rotate28(uint32_t x, int shift)
```

```
{
```

```
    return ((x << shift) |
```

```
            (x >> (28 - shift))) & 0xFFFFFFFF;
```

```
}
```

```
void generateKeys(uint64_t key,
```

```
                 uint64_t keys[16])
```

```
{
```

```
uint64_t k;
```

```
uint32_t C, D;
```

```
int i;
```

```
k = permute(key, PC1, 56, 64);
```

```
C = (k >> 28) & 0xFFFFFFFF;
```

```
D = k & 0xFFFFFFFF;
```

```
for(i = 0; i < 16; i++)
```

```
{
```

```
    C = rotate28(C, SHIFTS[i]);
```

```
    D = rotate28(D, SHIFTS[i]);
```

```
uint64_t combined =
```

```
    ((uint64_t)C << 28) | D;
```

```
keys[i] =
```

```
    permute(combined, PC2, 48, 56);
```

```
}
```

```
}
```

```
uint32_t feistel(uint32_t R,
```

```
    uint64_t key)
```

```
{
```

```
    uint64_t expanded;
```

```
    uint64_t x;
```

```
    uint32_t result = 0;
```

```
    int i;
```

```

expanded =
    permute((uint64_t)R, E, 48, 32);

x = expanded ^ key;

for(i = 0; i < 8; i++)
{
    int shift = 42 - i * 6;

    int bits =
        (x >> shift) & 0x3F;

    int row =
        ((bits & 0x20) >> 4) |
        (bits & 1);

    int col =
        (bits >> 1) & 15;

    result =
        (result << 4) |
        S[i][row][col];
}

return (uint32_t)
    permute(result, P, 32, 32);
}

```

```

uint64_t DES(uint64_t block,
              uint64_t keys[16],
              int decrypt)
{
    uint64_t x;
    uint32_t L, R;
    int i;

    x = permute(block, IP, 64, 64);

    L = x >> 32;
    R = x & 0xFFFFFFFF;

    for(i = 0; i < 16; i++)
    {
        uint32_t temp = R;

        int k =
            decrypt ? 15 - i : i;

        R =
            L ^ feistel(R, keys[k]);

        L = temp;
    }

    x =
        ((uint64_t)R << 32) | L;

```

```

    return permute(x, FP, 64, 64);
}

/* ----- DES MODES ----- */

/* ECB */
uint64_t ECB_Encrypt(uint64_t plaintext,
                     uint64_t keys[])
{
    return DES(plaintext, keys, 0);
}

uint64_t ECB_Decrypt(uint64_t ciphertext,
                     uint64_t keys[])
{
    return DES(ciphertext, keys, 1);
}

/* CBC */
uint64_t CBC_Encrypt(uint64_t plaintext,
                     uint64_t IV,
                     uint64_t keys[])
{
    return DES(plaintext ^ IV, keys, 0);
}

uint64_t CBC_Decrypt(uint64_t ciphertext,
                     uint64_t IV,
                     uint64_t keys[])

```

```

{
    return DES(ciphertext, keys, 1) ^ IV;
}

/* CFB */
uint64_t CFB_Encrypt(uint64_t plaintext,
                     uint64_t IV,
                     uint64_t keys[])
{
    return plaintext ^ DES(IV, keys, 0);
}

uint64_t CFB_Decrypt(uint64_t ciphertext,
                     uint64_t IV,
                     uint64_t keys[])
{
    return ciphertext ^ DES(IV, keys, 0);
}

/* OFB */
uint64_t OFB_Encrypt(uint64_t plaintext,
                     uint64_t IV,
                     uint64_t keys[])
{
    uint64_t output;

    output = DES(IV, keys, 0);

    return plaintext ^ output;
}

```

```
}
```

```
uint64_t OFB_Decrypt(uint64_t ciphertext,
```

```
    uint64_t IV,
```

```
    uint64_t keys[])
```

```
{
```

```
    uint64_t output;
```

```
    output = DES(IV, keys, 0);
```

```
    return ciphertext ^ output;
```

```
}
```

```
/* CTR */
```

```
uint64_t CTR_Encrypt(uint64_t plaintext,
```

```
    uint64_t counter,
```

```
    uint64_t keys[])
```

```
{
```

```
    return plaintext ^
```

```
        DES(counter, keys, 0);
```

```
}
```

```
uint64_t CTR_Decrypt(uint64_t ciphertext,
```

```
    uint64_t counter,
```

```
    uint64_t keys[])
```

```
{
```

```
    return ciphertext ^
```

```
        DES(counter, keys, 0);
```

```
}
```

```

/* ----- MAIN ----- */

int main()
{
    uint64_t plaintext;
    uint64_t key;
    uint64_t IV;
    uint64_t counter;

    uint64_t keys[16];

    uint64_t cipher;
    uint64_t decrypted;

    printf("DES MODES OF OPERATION\n");
    printf("-----\n");

    printf("Enter 16-digit hexadecimal plaintext: ");
    scanf("%llx",
        (unsigned long long *)&plaintext);

    printf("Enter 16-digit hexadecimal key: ");
    scanf("%llx",
        (unsigned long long *)&key);

    printf("Enter 16-digit hexadecimal IV: ");
    scanf("%llx",
        (unsigned long long *)&IV);

```

```
printf("Enter 16-digit hexadecimal counter: ");  
scanf("%llx",  
      (unsigned long long *)&counter);
```

```
generateKeys(key, keys);
```

```
/* ECB */
```

```
cipher =  
    ECB_Encrypt(plaintext, keys);
```

```
decrypted =  
    ECB_Decrypt(cipher, keys);
```

```
printf("\nECB");  
printf("\nCiphertext : %016llX",  
      (unsigned long long)cipher);  
printf("\nDecrypted : %016llX\n",  
      (unsigned long long)decrypted);
```

```
/* CBC */
```

```
cipher =  
    CBC_Encrypt(plaintext, IV, keys);
```

```
decrypted =  
    CBC_Decrypt(cipher, IV, keys);
```

```
printf("\nCBC");  
printf("\nCiphertext : %016llX",
```

```

        (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
        (unsigned long long)decrypted);

/* CFB */
cipher =
    CFB_Encrypt(plaintext, IV, keys);

decrypted =
    CFB_Decrypt(cipher, IV, keys);

printf("\nCFB");
printf("\nCiphertext : %016llX",
        (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
        (unsigned long long)decrypted);

/* OFB */
cipher =
    OFB_Encrypt(plaintext, IV, keys);

decrypted =
    OFB_Decrypt(cipher, IV, keys);

printf("\nOFB");
printf("\nCiphertext : %016llX",
        (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
        (unsigned long long)decrypted);

```

```

/* CTR */

cipher =

    CTR_Encrypt(plaintext, counter, keys);

decrypted =

    CTR_Decrypt(cipher, counter, keys);

printf("\nCTR");
printf("\nCiphertext : %016llX",
        (unsigned long long)cipher);
printf("\nDecrypted : %016llX\n",
        (unsigned long long)decrypted);

return 0;
}

```

OUTPUT:

The screenshot shows a C++ IDE with a file named `main.cpp`. The code implements a Counter (CTR) mode for encryption and decryption using a custom `CTR_Encrypt` and `CTR_Decrypt` functions. The program prompts the user for a 16-digit hexadecimal plaintext, key, IV, and counter. It then demonstrates encryption and decryption for three different modes: ECB, CBC, and CFB. The output pane shows the results of these operations, including the ciphertext and decrypted plaintext for each mode.

```

main.cpp
117 printf("mode ");
118 printf("\nCiphertext : %016llX",
119         (unsigned long long)cipher);
120 printf("\nDecrypted : %016llX\n",
121         (unsigned long long)decrypted);
122
123 /* CTR */
124 cipher =
125     CTR_Encrypt(plaintext, counter, keys);
126
127 decrypted =
128     CTR_Decrypt(cipher, counter, keys);
129
130 printf("\nCTR");
131 printf("\nCiphertext : %016llX",
132         (unsigned long long)cipher);
133 printf("\nDecrypted : %016llX\n",
134         (unsigned long long)decrypted);
135
136 return 0;
137 }

```

Output:

```

DES MODES OF OPERATION
-----
Enter 16-digit hexadecimal plaintext: 123456ABCD132536
Enter 16-digit hexadecimal key: 123456ABCD132536
Enter 16-digit hexadecimal IV: 1234567890ABCDEF
Enter 16-digit hexadecimal counter: 1234567890ABCDEF

ECB
Ciphertext : A10A07A1B1C22C03
Decrypted : 123456ABCD132536

CBC
Ciphertext : 4961AEBED3809C1A
Decrypted : 123456ABCD132536

CFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

OFB
Ciphertext : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

```

```
main.c
418 printf("\nCipherText : %016lX",
419        (unsigned long long)cipher);
420 printf("\nDecrypted : %016lX\n",
421        (unsigned long long)decrypted);
422
423 /* CTR */
424 cipher =
425     CTR_Encrypt(plaintext, counter, keys);
426
427 decrypted =
428     CTR_Decrypt(cipher, counter, keys);
429
430 printf("\nCTR");
431 printf("\nCipherText : %016lX",
432        (unsigned long long)cipher);
433 printf("\nDecrypted : %016lX\n",
434        (unsigned long long)decrypted);
435
436 return 0;
437 }
```

Output

```
CipherText : A10A07A1B1C22C03
Decrypted : 123456ABCD132536

ECB
CipherText : 4961AEBED3809C1A
Decrypted : 123456ABCD132536

CFB
CipherText : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

OFB
CipherText : A301F8DBD454DBF4
Decrypted : 123456ABCD132536

CTR
CipherText : A301F8DBD454DBF4
Decrypted : 123456ABCD132536
```

Code Execution Successful

RESULT:

Thus, the different modes of operation of DES—ECB, CBC, CFB, OFB, and CTR—were successfully implemented in C. Encryption and decryption were performed successfully, and the original plaintext was recovered for each mode.